

CSE 719 — Distributed & Cloud Computing

Lecture 4: Failure Detection & Group Membership

Question Analysis & Precise Solutions (2020–2024)

Dr. Atiqur Rahman · University of Chittagong

Exam: 1 July 2026

Contents

1	Reference: Core Concepts	2
1.1	Failures are the Norm	2
1.2	Two Sub-Protocols in Group Membership	2
1.3	Desirable Properties of a Failure Detector	2
1.4	Heartbeating Variants	2
1.5	SWIM Protocol (Scalable Weakly-consistent Infection-style Membership)	2
1.6	Gossip-Style Failure Detection Protocol	3
1.7	Suspicion Mechanism	3
1.8	Dissemination Options	3
2	2020 — Q2b (2.75 marks)	4
3	2022 — Q6b (2 marks)	6
4	2022 — Q6d (2 marks)	7
5	2024 — Q2d (3 marks)	8
6	2024 — Q3a (3 marks)	9
7	Summary Table	10

Reference: Core Concepts

Failures are the Norm

In a 12,000-server datacenter (each fails once per 10 years on average), the **Mean Time To Failure (MTTF)** of the next machine is ≈ 7.2 hours. Every distributed system must therefore include a failure detector.

Two Sub-Protocols in Group Membership

1. **Failure Detector (FD)**: detects which processes have crashed.
2. **Dissemination**: spreads failure/join/leave notifications to all members.

Desirable Properties of a Failure Detector

- **Completeness** — every failure is eventually detected (guaranteed).
- **Accuracy** — no mistaken (false-positive) detection. In practice: only probabilistic accuracy $PM(T)$.
- **Speed** — time until *some* process detects the failure.
- **Scale** — no bottlenecks; equal load per member; $O(1)$ per-node message load independent of group size N .

Key theorem [Chandra & Toueg 1996]: Completeness and Accuracy are *impossible to achieve simultaneously* in asynchronous, lossy networks. A perfect failure detector would solve the consensus problem — but FLP impossibility proves consensus is impossible in asynchronous networks with even one faulty process.

Heartbeating Variants

Variant	Mechanism	Problem
Centralized	All $\rightarrow$ one central monitor	Hotspot / single point of failure
Ring	Each node monitors its ring successor	Unpredictable with simultaneous failures
All-to-All	Every node $\rightarrow$ every other node	$O(N)$ message load per node per period
Gossip-style	Random subset gossip; merge lists	Good accuracy; $O(N \log N/T)$ load
SWIM	Ping; indirect via K nodes if no ack	Constant detection time; $O(1)$ load

SWIM Protocol (Scalable Weakly-consistent Infection-style Membership)

1. Process p_i picks a random target p_j and sends a **ping**.
2. If no **ack** within protocol period T' , send **ping-req** to K random processes.
3. Those K processes each send a **ping** to p_j and relay any **ack** back to p_i .
4. If p_i still receives no ack after the full period: mark p_j as **Suspected**.
5. After timeout T_{cleanup} : declare p_j **Failed**.

SWIM properties: first detection time $= T' \cdot \lceil e/(e-1) \rceil$ (constant, independent of N); process load $= O(1)$ per period; false positive rate tunable via K .

Gossip-Style Failure Detection Protocol

1. Each member maintains a **membership list**: array of $\langle \text{address}, \text{heartbeat-counter}, \text{local-time} \rangle$ for each known member.
2. Periodically (every gossip period t_g), each member picks b random targets and sends its full membership list to them.
3. On receipt, lists are **merged** (keep the higher heartbeat counter for each entry).
4. If a member's heartbeat counter has not increased for more than T_{fail} seconds $\Rightarrow$ mark as **failed**.
5. After T_{cleanup} seconds, delete the entry to prevent re-introduction of the stale (failed) entry from other nodes.

Suspicion Mechanism

To reduce false positives, a process is first *suspected* before being declared failed.



Each process has a per-process **incarnation number** (incremented only by itself when it receives a Suspect message about itself). Priority rule: (Failed, inc#) overrides everything; within the same inc#: (Suspect) > (Alive).

Dissemination Options

- **Multicast (IP)**: unreliable; multiple simultaneous multicasts.
- **Point-to-point (TCP/UDP)**: reliable but expensive.
- **Infection-style (best)**: piggyback membership updates on existing FD messages (pings and acks) — zero extra messages.

Q2(b): What are the techniques for fault detection and recovery in a distributed operating system? [2.75 marks]

Part 1: Fault Detection Techniques

In a distributed OS, every process periodically broadcasts a *heartbeat* signal so that other processes can detect its failure if the heartbeat stops. Four main techniques exist:

1. **Centralized Heartbeating.** All N nodes send periodic heartbeats to one designated monitor process p_j . If p_j does not receive a heartbeat from p_i within a timeout, it marks p_i as failed. *Problem:* p_j is a hotspot (single point of failure; $O(N)$ load).
2. **Ring Heartbeating.** Nodes are arranged in a logical ring; each node monitors only its immediate successor. *Problem:* unpredictable behavior when multiple nodes fail simultaneously — successor and predecessor can both be failed.
3. **All-to-All Heartbeating.** Every node sends heartbeats to every other node. Benefit: equal load per member; no single point of failure. *Problem:* $O(N)$ messages per member per period — does not scale.
4. **Gossip-Style Heartbeating (robustness improvement over all-to-all).** Each node maintains a membership list with a heartbeat counter per member. Periodically, each node selects a random subset of members and sends the entire list (gossip). Recipients merge received lists with their own. A member whose counter has not increased for T_{fail} seconds is marked as failed; after T_{cleanup} , the entry is deleted. Benefit: failure information propagates along multiple independent random paths, so packet loss on one path does not prevent detection. *Real use:* Amazon EC2/S3 (rumored).

Part 2: Recovery Techniques

Once a failure is detected, recovery involves two steps:

1. **Dissemination.** The detecting process spreads the failure notification to all group members via:
 - Multicast (fast but unreliable), or
 - Infection-style: membership updates piggybacked onto existing heartbeat/ping messages (zero additional network cost).
2. **Recovery actions by the application.** After all members update their membership lists:
 - Re-assign the failed process's tasks to surviving processes.
 - Re-replicate any data blocks held by the failed node to restore redundancy.
 - Update routing tables, load-balancer entries, or Paxos leader election as appropriate for the system.

False-Positive Reduction (Suspicion Mechanism). Before declaring a process definitively failed, a process is first *suspected* (Alive $\rightarrow$ Suspected $\rightarrow$ Failed) with a grace period for the node to refute the suspicion by incrementing its incarnation number. This prevents unnecessary re-assignment caused by transient network congestion.

Exam pattern: 2020 Q2b (2.75 marks) is the only direct "techniques for FD and recovery" question in the set. Expect ~ 1 mark for detection techniques, ~ 1 mark for recovery actions, ~ 0.75 mark for the improvement/suspicion point.

Q6(b): How to increase the robustness of all-to-all heartbeating?

[2 marks]

Problem with All-to-All Heartbeating. In all-to-all heartbeating, each of N members sends heartbeats directly to all $N - 1$ other members every period T , giving a per-member message load $L = N/T$. A single dropped heartbeat can cause a false-positive failure detection, and the $O(N)$ per-node load makes this scheme hard to scale.

Solution: Switch to Gossip-Style Heartbeating.

1. Each member maintains a **membership list**: an array of $\langle \text{address}, \text{heartbeat-counter} \rangle$ for all known members.
2. Periodically, each member picks a **random subset** of members and sends the entire list to that subset (gossip message).
3. Recipients **merge** the incoming list with their own (keep the higher heartbeat counter for each entry).
4. A member whose counter has not increased for T_{fail} seconds is marked **failed**; after T_{cleanup} it is removed.

Why this increases robustness:

- Failure information propagates through *multiple independent random paths*. Even if some gossip messages are lost, the information still reaches all nodes via other paths (infection-style spreading).
- No single link failure can prevent detection of a crashed node.
- Provides *good accuracy properties*: the heartbeat counter approach is tolerant of occasional packet loss without falsely declaring nodes failed.
- Message load: $O(N \log N/T)$ — higher than all-to-all per round but far more robust, and the gossip period can be tuned.

Exam pattern: 2022 Q6b (2 marks). Answer must name **gossip-style heartbeating** as the solution and explain *why* it is more robust (multiple paths, merge-on-receipt). Two sentences on the protocol + two on the robustness justification = full marks.

Q6(d): How do you design a group membership protocol?

[2 marks]

A group membership protocol must maintain a consistent (or almost-consistent) view of which processes are currently alive. It is built from **two separated sub-protocols**:

Sub-protocol I — Failure Detector

- Detects which members have crashed.
- Best-practice choice: **SWIM** (Scalable Weakly-consistent Infection-style Membership):
 1. Send a direct **ping** to a random target p_j .
 2. If no **ack** arrives within period T' , send **ping-req** to K random other members, asking each to **ping** p_j on your behalf.
 3. If no indirect **ack** arrives either: mark p_j as **Suspected**.
 4. After timeout T_{cleanup} : declare p_j **Failed**.
- SWIM achieves *constant* detection time (independent of group size N) and $O(1)$ per-member message load.

Sub-protocol II — Dissemination

- Spreads Suspected/Failed/Join notifications to every member.
- Best-practice: **Infection-style (zero extra messages)**: piggyback membership updates (join/fail/suspect events) onto existing SWIM **ping** and **ack** messages. Updates spread in $O(\log N)$ rounds (epidemic propagation).

Additional Design Elements

- **Suspicion mechanism**: Alive $\rightarrow$ Suspected $\rightarrow$ Failed state machine reduces false positives from transient packet loss.
- **Incarnation numbers**: Each process holds a counter incremented only by itself (e.g., on receiving a Suspect message about itself). Higher incarnation number overrides lower; (Failed, inc#) overrides everything. Prevents stale gossip from re-introducing a failed entry.

Exam pattern: 2022 Q6d (2 marks). Key structure: name the two sub-protocols (Failure Detector + Dissemination), give a concrete mechanism for each (SWIM + infection-style), and mention the suspicion mechanism. That is the complete answer.

Q2(d): What is the primary goal of failure detection in distributed systems? Why is perfect and instantaneous failure detection practically impossible in asynchronous networks? [3 marks]

Primary Goal — Completeness. The primary goal of failure detection is **Completeness**: every process failure must eventually be detected by at least one non-faulty process, so the system can take recovery actions (re-assign tasks, re-replicate data, trigger a new leader election). Without completeness, a silent crashed process may block the entire system indefinitely.

Why Perfect and Instantaneous Detection is Impossible.

A *perfect* failure detector would satisfy both:

- **Completeness:** every failure is detected.
- **Accuracy:** no alive process is ever mistakenly declared failed.

In an **asynchronous network**, message delivery delay is *unbounded* and messages may be lost. This creates a fundamental impossibility:

1. Process p_i sends a heartbeat/ping to p_j and waits for a response.
2. p_i cannot distinguish between two scenarios:
 - p_j has *crashed* (no response ever), or
 - p_j is *alive* but the message is *delayed or lost*.
3. If p_i sets a timeout T_{fail} and declares p_j failed on expiry:
 - A too-short timeout triggers on a slow-but-alive $p_j \Rightarrow$ false positive $\Rightarrow$ **violates Accuracy**.
 - A too-long timeout delays detection $\Rightarrow$ **violates instantaneousness**.
 - Any fixed timeout can be exceeded by a sufficiently large (but finite) network delay $\Rightarrow$ no timeout can be simultaneously correct and fast.

Formal result [Chandra & Toueg, 1996]: Completeness and Accuracy *cannot be achieved simultaneously* in asynchronous lossy networks. A perfect failure detector would enable solving the consensus problem, but the **FLP impossibility theorem** proves consensus is impossible in asynchronous networks with even one possible faulty process.

Practical compromise: Real systems guarantee **Completeness** (every failure detected) but offer only **probabilistic Accuracy** ($PM(T)$ = probability of a mistaken detection within time T). The false-positive rate and detection time are tunable via timeouts and redundancy parameters (e.g., SWIM's K indirect pingers).

Exam pattern: 2024 Q2d (3 marks). Award breakdown: ~ 1 mark for Completeness definition, ~ 1 mark for the distinguish-crash-vs-delay argument, ~ 1 mark for the Chandra & Toueg impossibility result + practical implication. Keyword the examiner wants: **asynchronous network, unbounded delay, Completeness + Accuracy impossible, probabilistic accuracy**.

Q3(a): A server is carrying out requests from many clients. In some situations a server cannot bound the time that it will take to execute a request. Why? What should it do in order to execute the request within a bounded time? [3 marks]

Part 1: Why the Server Cannot Bound Response Time.

In a distributed system that uses an **asynchronous network model**, several factors make it impossible to bound execution time:

1. **Unbounded network delay.** There is no fixed upper bound on message transmission time. A request from a remote client can be arbitrarily delayed before arriving at the server, or a response can be delayed before reaching the client.
2. **Message loss.** Packets can be dropped (network congestion, hardware fault). The server (or client) cannot know whether to wait longer or retry.
3. **OS scheduling non-determinism.** The server process can be preempted by the OS scheduler at any time. CPU time is shared with other processes, so execution time is not deterministic.
4. **No global clock.** In an asynchronous system, clocks are not synchronized. The server cannot measure how much "wall-clock time" a client has been waiting without a shared reference.

Part 2: What the Server Should Do to Execute Within Bounded Time.

Since perfect bounded response time is impossible in an asynchronous network, the server should adopt *timeout-based* and *probabilistic* mechanisms:

1. **Set a timeout T_{fail} .** If no response (or no heartbeat from the client) is received within T_{fail} , *suspect* the client or the pending request. This is the only practical way to bound waiting time.
2. **Use indirect detection (SWIM-style).** Before declaring the client failed, ask K other nodes to contact the client on the server's behalf. Confirmation from multiple paths reduces false positives caused by a single congested link.
3. **Adopt the Suspicion Mechanism.** Transition through states: Alive $\rightarrow$ Suspected $\rightarrow$ Failed, with $T_{\text{cleanup}} > T_{\text{fail}}$. This buffers against transient delays causing premature failure declarations.
4. **Accept probabilistic accuracy.** The server should acknowledge that any timeout may occasionally fire on a slow-but-alive client (false positive) and design recovery actions (idempotent request re-execution, exactly-once semantics) accordingly.

Key principle: The server trades *perfect accuracy* for *guaranteed completeness and bounded waiting time*, tuning T_{fail} to achieve an acceptable false-positive rate $PM(T)$.

Exam pattern: 2024 Q3a (3 marks). Two clear parts — examiner expects: (1) at least 2 concrete reasons for unbounded time (async delay + message loss are the core ones), (2) timeout-based approach as the answer, with mention of suspicion mechanism and probabilistic accuracy. This question is directly about the FLP/async impossibility applied to the server side.

Summary Table

Year	Q	Marks	Topic	Key answer
2020	Q2b	2.75	FD techniques + recovery	4 heartbeat variants + dissemination + suspicion
2022	Q6b	2	All-to-all robustness	Gossip-style: random subset + merge
2022	Q6d	2	Group membership design	SWIM (FD) + infection-style (dissemination)
2024	Q2d	3	FD goal + impossibility	Completeness goal; async $\Rightarrow$ C+A impossible
2024	Q3a	3	Server bounded time	Async delays; timeout + suspect + probabilistic

Yield 3/5. With 2024 adding two questions, this topic is *low-risk skip only if time is very tight*. The 2024 Q2d and Q3a answers share the same core argument (async impossibility, timeout-based approach) — learn once, answer twice.

Key terms to write in exam: Completeness, Accuracy, asynchronous network, unbounded delay, gossip-style, Chandra & Toueg, SWIM, T_{fail} , T_{cleanup} , suspicion mechanism, incarnation number, infection-style dissemination.